Table of Contents

ugajapa-translation-api

Project	UgaJapa Connect — Akademia Japan Co., Ltd.
Repository	ugajapa-translation-api (new)
Platform	Mattermost TransChecker Plugin
Authors	Translation Group — Makerere University
Version	v1.0.0 | July 2026
Status	Ready for Implementation

1 Executive Summary

The **ugajapa-translation-api** repository is the central backend service of the UgaJapa Connect platform. It acts as the single point of contact for all translation requests from any plugin — Rocket.Chat, Mattermost, or any future integration — while managing user accounts, per-user API keys, usage metering, and monthly billing.

Rather than calling external translation providers directly, this service routes all requests through its own internally-hosted translation engine: the **UgaJapa Bot**, powered by Facebook's NLLB-200 model (No Language Left Behind), which supports 200 languages including English, Japanese, Luganda, Acholi, Runyankole, and Ateso — languages that mainstream APIs do not optimise for.

Every translation response includes a **quality evaluation score** computed via back-translation (Levenshtein + semantic similarity), a feature that differentiates this API from standard translation services and satisfies Kuwatani-san's key requirement.

Feature	Description
Own Translation Engine	NLLB-200 locally hosted — no per-call fees to Google or NVIDIA
Ugandan Language Support	Luganda, Acholi, Runyankole, Ateso — natively in NLLB-200
Quality Evaluation	Every response includes Levenshtein + semantic back-translation score
Per-User API Keys	Each user gets unique keys (ugj_live_xxx) for authentication
Usage Metering	Characters translated recorded per key per user per month
Monthly Billing	Automated usage-based billing calculation per plan tier
Swappable Engines	Router architecture — add/switch engines with one code change
Mattermost Integration	Plugin calls this API; plugin code needs no changes to the translation logic

2 Repository Overview & Structure

The **ugajapa-translation-api** repository contains two sub-services that work together as a single deployable unit:

- **Translation API Service** (Node.js / Express / TypeScript) — handles auth, key management, request routing, quality scoring, usage recording, and billing. Runs on port 5000.
- **UgaJapa Bot** (Python / FastAPI) — hosts the NLLB-200 translation model locally. Called internally by the Translation API. Runs on port 8000. Never exposed to the public directly.

Folder Structure

```
ugajapa-translation-api/ ← this repository
■
■■■ translation-api/ ← Node.js service (port 5000)
■ ■■■ src/
■ ■ ■■■ index.ts ← Express app entry point + all routes
■ ■ ■■■ db.ts ← PostgreSQL connection pool
■ ■ ■■■ auth.ts ← Signup, login, JWT management
■ ■ ■■■ keys.ts ← API key generation, validation, revocation
■ ■ ■■■ translate.ts ← Core translate + back-translation scoring
■ ■ ■■■ router.ts ← Engine selection (bot / Sunbird / fallback)
■ ■ ■■■ ugajapa-bot.ts ← HTTP client for the Python bot
■ ■ ■■■ billing.ts ← Usage recording + monthly bill calculation
■ ■ ■■■ usage.ts ← Character counting + quota checks
■ ■■■ .env.example ← Environment variable template
■ ■■■ package.json
■ ■■■ tsconfig.json
■
■■■ ugajapa-bot/ ← Python service (port 8000)
■ ■■■ server.py ← FastAPI app with NLLB-200 model
■ ■■■ requirements.txt
■
```

```
■■■ docker-compose.yml ← PostgreSQL + both services

■■■ README.md

■■■ .env.example
```

3 System Architecture

The full system spans three repositories. This document covers the **ugajapa-translation-api** repository (Layers 2 and 3). The Mattermost plugin (Layer 1) is managed separately in its own repository.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ LAYER 1 — Mattermost Plugin (ugajapa-mattermost-plugin repo) ■
■ Go server + React webapp — runs inside Mattermost Docker container ■
■ User sends message → plugin calls Translation API with X-API-Key ■
■ Plugin receives response → displays translation + quality badge ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ POST /translate
■ Header: X-API-Key: ugj_live_abc123
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ LAYER 2 — Translation API Service (this repository — port 5000) ■
■ ■
■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■ Auth Layer ■ ■ Key Manager ■ ■ Usage & Billing Engine ■ ■
■ ■ /auth/* ■ ■ /keys/* ■ ■ /usage/* /billing/* ■ ■
■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■ Translation Orchestrator ■ ■
■ ■ 1. Validate API key (PostgreSQL lookup) ■ ■
■ ■ 2. Check quota (plan limit not exceeded) ■ ■
■ ■ 3. Route to correct engine (router.ts) ■ ■
■ ■ 4. Run quality scoring (back-translation) ■ ■
■ ■ 5. Record usage (usage_records table) ■ ■
■ ■ 6. Return full response with quality object ■ ■
```


Luganda pairs All other pairs
Sunbird AI
(external)
UgaJapa Bot
port 8000
POST /translate (internal only)
LAYER 3 — UgaJapa Bot (this repository — port 8000)
Python / FastAPI + facebook/nllb-200-distilled-600M
Supports: EN JA LG (Luganda) ACH (Acholi) SW FR + 194 more
Never called directly by plugins — internal to this repo only

4 The Three Repositories

As directed by Kuwatani-san, the codebase is split into three independent repositories, each independently deployable and versioned:

Repository	Language / Stack	Port	Purpose	Status
ugajapa-translation-api	Node.js + Python	5000 / 8000	Central API service + UgaJapa Bot	NEW — this document
ugajapa-rocketchat-plugin	TypeScript (Apps Engine)	3000	Rocket.Chat translation plugin	Existing
ugajapa-mattermost-plugin	Go + React/TypeScript	8065	Mattermost TransChecker plugin	NEW — separate repo

✓ The plugins (Rocket.Chat and Mattermost) are clients of the Translation API. They send requests to it but contain no translation logic themselves. This means the translation engine can be upgraded, swapped, or improved without touching either plugin.

5 UgaJapa Bot — Own Translation Engine

The UgaJapa Bot is a lightweight Python FastAPI service that hosts Facebook's NLLB-200 (No Language Left Behind) model locally. It is the primary translation engine called by the Translation API service. It is never called directly by any plugin.

5.1 Why NLLB-200

- Supports 200 languages including **Luganda, Acholi, Runyankole, and Ateso** — mainstream engines do not.
- Open source and free to run — no per-character API fees after initial setup.
- Runs entirely on your own server — data never leaves your infrastructure.
- Quality is comparable to commercial APIs for English ↔ Japanese translation.
- Future fine-tuning on Uganda-Japan domain data will improve quality over time.

5.2 Model Options

Model	RAM Needed	Quality	Recommendation
nllb-200-distilled-300M	~2 GB	Good	Use if RAM < 4 GB
nllb-200-distilled-600M	~4 GB	Better	Recommended starting point
nllb-200-distilled-1.3B	~8 GB	Best	Use if server has enough RAM

5.3 Bot Service Code (server.py)

```
from fastapi import FastAPI, HTTPException

from pydantic import BaseModel

from transformers import pipeline

import torch


app = FastAPI(title='UgaJapa Translation Bot')


LANG_CODES = {

'en': 'eng_Latn', # English

'ja': 'jpn_Jpan', # Japanese

'lg': 'lug_Latn', # Luganda

'fr': 'fra_Latn', # French

'sw': 'swh_Latn', # Swahili
```

```
'ach': 'ach_Latn', # Acholi
}

translator = pipeline('translation',
model='facebook/nllb-200-distilled-600M',
device=0 if torch.cuda.is_available() else -1)

class TranslateRequest(BaseModel):
text: str
from_lang: str
to_lang: str

@app.get('/health')
def health(): return {'status': 'ok', 'engine': 'nllb-200-distilled-600M'}

@app.post('/translate')
def translate(req: TranslateRequest):
if req.from_lang == req.to_lang:
return {'translated': req.text, 'engine': 'none'}
result = translator(req.text,
src_lang=LANG_CODES[req.from_lang],
tgt_lang=LANG_CODES[req.to_lang], max_length=512)
return {'translated': result[0]['translation_text'],
'engine': 'nllb-200-distilled-600M'}
```

5.4 Start Commands

```
# Install dependencies (once)
pip install fastapi uvicorn transformers torch sentencepiece

# Start (first run downloads ~1.2 GB model)
cd ugajapa-bot
```

```
uvicorn server:app --host 0.0.0.0 --port 8000


# Verify

curl http://localhost:8000/health

# → {"status":"ok","engine":"nllb-200-distilled-600M"}
```

6 Translation API Service — Full Feature Reference

6.1 User Authentication

Users register once through **POST /auth/signup** and receive a JWT session token. This token is used only for account management (key generation, usage viewing, billing). It is never used for translation requests directly.

6.2 API Key System

After signing up and logging in, each user generates one or more API keys. These keys are what the Mattermost plugin (and any other client) uses in the **X-API-Key** header on every translation request.

- Keys are prefixed **ugj_live_** for production or **ugj_test_** for sandbox.
- The key value is shown **once only** at generation time — only a bcrypt hash is stored in the database.
- A user can have multiple keys — one per plugin installation, one per company integration.
- Keys can be revoked instantly. Revoked keys return 401 Unauthorized immediately.
- Each key tracks its own last_used timestamp for audit purposes.

6.3 Translation with Quality Evaluation

Every call to **POST /translate** returns not just the translated text but a complete quality evaluation object. This is the feature that differentiates the UgaJapa API from standard translation APIs.

How quality scoring works:

```
Step 1: Forward translation

text (EN) → UgaJapa Bot → translated (JA)


Step 2: Back-translation

translated (JA) → UgaJapa Bot → reversed (EN)


Step 3: Compare original vs reversed

Levenshtein score = character-level similarity (0.0 to 1.0)

Semantic score = word-overlap F1 (0.0 to 1.0)


Step 4: Assign label

score >= 0.70 → Good (green) — high confidence

score >= 0.40 → Fair (amber) — acceptable

score < 0.40 → Low (red) — accuracy warning shown
```

6.4 Engine Routing

The router selects which engine to use based on the language pair. This design means swapping or adding engines requires only a change to router.ts — no plugin changes needed.

Language Pair	Primary Engine	Fallback
English ↔ Japanese	UgaJapa Bot (NLLB-200)	Google (if confidence < 0.4)
English ↔ Luganda	Sunbird AI	UgaJapa Bot
Japanese ↔ Luganda	English pivot via UgaJapa Bot	UgaJapa Bot direct
English ↔ French/Swahili	UgaJapa Bot (NLLB-200)	None
English ↔ Acholi	UgaJapa Bot (NLLB-200)	Sunbird AI
Any other pair	UgaJapa Bot (NLLB-200)	Google (optional)

6.5 Usage Metering

Every successful translation call records a row in the **usage_records** table containing: the API key used, user ID, character count, source/target languages, engine used, quality score, and timestamp. This data feeds both the billing calculation and the analytics dashboard.

6.6 Monthly Billing

At the end of each calendar month, a billing record is generated per user by summing all usage_records for that month and calculating the amount due based on the user's plan. Users can retrieve their bill via **GET /billing/:month**.

7 API Endpoint Reference

7.1 Authentication

Method	Endpoint	Auth Required	Description
POST	/auth/signup	None	Create a new user account. Returns JWT token.
POST	/auth/login	None	Login with email + password. Returns JWT token.
POST	/auth/logout	Bearer JWT	Invalidate the current session token.

POST /auth/signup — Request / Response

```
// Request body

{ "email": "user@company.com", "password": "SecurePass123!", "full_name": "Emuduko Bernard" }


// Response 201

{ "user_id": "uuid", "email": "user@company.com", "token": "eyJhb...", "message": "Account created" }
```

7.2 API Key Management

Method	Endpoint	Auth Required	Description
POST	/keys/generate	Bearer JWT	Issue a new API key for the user.
GET	/keys	Bearer JWT	List all active keys (values masked).
DELETE	/keys/:key_id	Bearer JWT	Revoke a specific API key immediately.

POST /keys/generate — Request / Response

```
// Request body

{ "name": "Mattermost Plugin Key" }


// Response 201

{

"key_id": "uuid",

"key_value": "ugj_live_abc123xyz789", ← shown ONCE only

"name": "Mattermost Plugin Key",
```

```
"created_at": "2026-07-08T10:00:00Z"

}
```

7.3 Translation

■ This is the primary endpoint called by the Mattermost plugin on every message translation. The plugin already sends requests to this endpoint — only the internal engine changes.

Method	Endpoint	Auth Required	Description
POST	/translate	X-API-Key header	Translate text with quality evaluation.
POST	/detect	X-API-Key header	Detect the language of a text string.
GET	/languages	X-API-Key header	List all supported language codes.
GET	/health	None	Service health check.

POST /translate — Full Request / Response

```
// Request headers

X-API-Key: ugj_live_abc123xyz789

Content-Type: application/json


// Request body

{ "text": "Good morning everyone", "from": "en", "to": "ja" }


// Response 200

{

"origin": "Good morning everyone",

"translated": "■■■■■■■■■■■■■■■",

"reversed": "Good morning, everyone",

"from": "en",

"to": "ja",

"engine": "ugajapa-bot",

"quality": {

"levenshtein": 0.91,
```

```
"semantic": 0.88,
"label": "Good",
"color": "green"
},
"characters_used": 22,
"cultural_hint": null
}
```


7.4 Usage & Billing

Method	Endpoint	Auth Required	Description
GET	/usage/summary	Bearer JWT	Characters used this month + quota status.
GET	/usage/history	Bearer JWT	Daily usage breakdown for the current month.
GET	/billing/:month	Bearer JWT	Bill for a specific month (YYYY-MM format).
GET	/billing	Bearer JWT	Full billing history across all months.

7.5 Admin (Internal — Akademia Japan)

Method	Endpoint	Description
GET	/admin/users	List all registered users with plan and usage totals.
GET	/admin/usage	Total API usage across all users this month.
GET	/admin/usage/:user_id	Per-user usage breakdown.
POST	/admin/users/:id/plan	Upgrade or downgrade a user's plan.
GET	/admin/billing/summary	Total revenue across all users for a given month.
DELETE	/admin/users/:id	Deactivate a user account.

8 Database Schema

PostgreSQL is used as the database, running in the same Docker stack as Mattermost. A separate database called **ugajapa_api** is created inside the same PostgreSQL container.

Table Relationships

```
users (1) ■■■■■■■■■■■■■■■■■■■■■■ (many) api_keys

■

users (1) ■■■■■■■■■■■■■■■■■■■■■■ (many) usage_records

■

api_keys (1) ■■■■■■■■■■■■■■■■■■ (many) usage_records


users (1) ■■■■■■■■■■■■■■■■■■■■■■ (many) billing
```

Table: users

Column	Type	Description
id	UUID PK	Auto-generated primary key
email	VARCHAR UNIQUE	User's email address — login identifier
password_hash	VARCHAR	bcrypt hash — plain password never stored
full_name	VARCHAR	Display name
plan	VARCHAR	free | starter | business | enterprise
created_at	TIMESTAMP	Account creation time
active	BOOLEAN	FALSE = account suspended

Table: api_keys

Column	Type	Description
id	UUID PK	Auto-generated primary key
user_id	UUID FK → users	Owner of this key
key_value	VARCHAR UNIQUE	Hashed key (ugj_live_xxx shown once at creation)
name	VARCHAR	User-defined label e.g. 'Mattermost Plugin Key'
created_at	TIMESTAMP	When key was generated
last_used	TIMESTAMP	Last successful API call with this key
revoked_at	TIMESTAMP	NULL = active. Set to revoke the key.

Table: usage_records

Column	Type	Description
id	UUID PK	Auto-generated primary key
api_key_id	UUID FK → api_keys	Which key made this request
user_id	UUID FK → users	Which user owns the key
characters	INTEGER	Number of characters in the translated text
from_lang	VARCHAR	Source language code (en, ja, lg, ...)
to_lang	VARCHAR	Target language code
engine	VARCHAR	ugajapa-bot | sunbird | google-fallback
quality_score	DECIMAL(4,3)	Levenshtein score from back-translation
timestamp	TIMESTAMP	When the translation was made

Table: billing

Column	Type	Description
id	UUID PK	Auto-generated primary key
user_id	UUID FK → users	Which user this bill belongs to
month	VARCHAR(7)	Format: 2026-07
characters_total	BIGINT	Total characters translated that month
amount_usd	DECIMAL(10,2)	Calculated bill amount in USD
paid	BOOLEAN	Payment status
generated_at	TIMESTAMP	When the bill was calculated

9 Feature Interactions — End-to-End Flows

9.1 New User Sign-Up and First Translation

This is the complete journey for a brand new company admin or individual user from zero to their first translated message in Mattermost.

```
STEP 1 — User signs up

User → POST /auth/signup { email, password, full_name }

API → creates user row in users table

API → returns JWT token


STEP 2 — User generates an API key

User → POST /keys/generate { name: 'Mattermost Plugin Key' }

→ Header: Authorization: Bearer

API → generates ugj_live_abc123xyz789

API → stores bcrypt hash in api_keys table

API → returns key_value ONCE (user must copy it now)


STEP 3 — Admin pastes key into Mattermost plugin settings

Mattermost → System Console → Plugin Settings

→ Translation API Key: ugj_live_abc123xyz789

→ Translation API URL: http://localhost:5000

→ Save


STEP 4 — User sends a message in Mattermost

User types: 'Good morning everyone'

User clicks ■ → Translate message


STEP 5 — Plugin calls Translation API

Plugin (Go) → POST http://localhost:5000/translate

→ Header: X-API-Key: ugj_live_abc123xyz789
```

```
→ Body: { text, from: 'en', to: 'ja' }

STEP 6 — Translation API processes the request
→ Validates key against api_keys table ✓
→ Checks quota: user is on free plan, 22 chars used of 50,000 ✓
→ Routes to UgaJapa Bot (EN→JA, not a Luganda pair)
→ Calls POST http://localhost:8000/translate
→ Receives: { translated: '[redacted]', engine: 'nllb-200' }
→ Back-translates JA→EN via bot
→ Scores: levenshtein=0.91, semantic=0.88, label='Good'
→ Records: usage_records row (22 chars, en, ja, ugajapa-bot, 0.91)
→ Returns full response to plugin

STEP 7 — Plugin displays result in Mattermost
→ Translation shown below original message
→ Green 'Good' quality badge displayed
→ Characters used updated in user's quota
```

9.2 Luganda Translation (Sunbird AI Routing)

```
User types a Luganda message: 'Wasuze otya nno' (How are you?)
Plugin calls POST /translate { text, from: 'lg', to: 'ja' }

Translation API router.ts detects source lang = 'lg'
→ Routes to Sunbird AI (specialist for Ugandan languages)
→ Sunbird translates lg → en (English pivot)
→ UgaJapa Bot translates en → ja
→ Back-translation scored as usual
→ Response engine field: 'sunbird+ugajapa-bot'

If Sunbird AI is unavailable:
```

```
→ Falls back to UgaJapa Bot direct (NLLB-200 has Luganda support)

→ Response engine field: 'ugajapa-bot-fallback'
```

9.3 Quota Exceeded Flow

```
User on Free plan has used 49,980 of 50,000 characters

User sends a message with 25 characters


Translation API → checks quota

→ 49,980 + 25 = 50,005 > 50,000 limit

→ Returns HTTP 429 Too Many Requests:

{ "error": "Monthly quota exceeded",

"used": 49980, "limit": 50000, "plan": "free",

"upgrade_url": "https://api.ugajapa.ac.ug/upgrade" }


Mattermost plugin receives 429

→ Displays message to user:

'Translation quota reached for this month.

Please upgrade your plan to continue.'
```

9.4 API Key Revocation Flow

```
Admin detects a key has been exposed (e.g. committed to GitHub)


Admin → DELETE /keys/:key_id

→ Header: Authorization: Bearer

API → sets revoked_at = NOW() in api_keys table


Any plugin still using the old key:

→ POST /translate with revoked key

→ API checks: revoked_at IS NOT NULL

→ Returns HTTP 401 Unauthorized:
```

```
{ "error": "API key has been revoked" }

Admin generates a new key → paste into plugin settings
→ Translations resume immediately
```

9.5 Monthly Billing Calculation Flow

```
On 1st of each month — automated cron job runs:

SELECT user_id, SUM(characters) as total
FROM usage_records
WHERE timestamp >= '2026-07-01' AND timestamp < '2026-08-01'
GROUP BY user_id

For each user:
→ Look up their plan (free / starter / business / enterprise)
→ Calculate amount_usd based on plan pricing
→ Insert row into billing table

User can then call:
GET /billing/2026-07
→ { "month": "2026-07", "characters_total": 48230,
"plan": "starter", "amount_usd": 9.00, "paid": false }
```

9.6 Voice Message Translation Flow (Mattermost)

```
User records a voice message in Mattermost
Recipient clicks 'Translate to text' button

Mattermost plugin (Go server):
→ Sends audio to STT pipeline
If EN/JA → Google Speech-to-Text
If LG/ACH → Groq Whisper → Sunbird AI STT
```

```
→ Receives transcript: 'Good morning everyone'

Plugin calls POST /translate (same as text flow):

→ Header: X-API-Key: ugj_live_abc123

→ Body: { text: 'Good morning everyone', from: 'en', to: 'ja' }

Translation API processes exactly as text flow:

→ UgaJapa Bot translates

→ Quality scored

→ Usage recorded (characters of transcript, not audio)

→ Response returned to plugin

Plugin displays:

→ WhatsApp-style audio player

→ Translated transcript below player

→ Quality badge

→ SRT download button (for video messages)
```

10 Pricing & Plan Model

The service uses a usage-based billing model measured in characters translated per month. One character = one character of the input text (not the translation output).

Plan	Characters / Month	Price (USD)	Overage	Best For
Free	50,000	$0.00	Blocked	Testing / individuals
Starter	500,000	$9.00	$0.50 per 10K	Small teams
Business	5,000,000	$49.00	$0.30 per 10K	Companies
Enterprise	Unlimited	Custom	Included	Large organisations

! For the internship evaluation phase, all team members use Free plan keys. Upgrade tiers are proposed for the production commercial launch.

11 Security Model

Layer	Mechanism	Detail
Password storage	bcrypt hashing	Plain passwords never stored — bcrypt hash only
Session tokens	JWT (HS256)	Signed with JWT_SECRET from .env — 24h expiry
API keys	bcrypt hash	Key shown once at generation — only hash in DB
Transport	HTTPS (production)	All traffic encrypted via TLS/Let's Encrypt
Key prefix	ugj_live_ / ugj_test_	Visual distinction between production and test keys
Revocation	Instant DB flag	revoked_at field checked on every request
Bot access	Internal only	Port 8000 never exposed to public — localhost only
Admin endpoints	Admin JWT	Separate admin token required — not same as user JWT
Rate limiting	Per-key throttle	Max 10 requests/second per API key

12 Deployment Guide

12.1 Local Development (Current)

```
# Terminal 1 — PostgreSQL (already running with Mattermost)

cd ~/Desktop/MATTERMOST/MATTERMOST_PLUGIN

docker compose up -d


# Terminal 2 — UgaJapa Bot

cd ~/Desktop/MATTERMOST/ugajapa-translation-api/ugajapa-bot

uvicorn server:app --host 0.0.0.0 --port 8000


# Terminal 3 — Translation API

cd ~/Desktop/MATTERMOST/ugajapa-translation-api/translation-api

npx tsx --env-file=.env src/index.ts


# Verify all services

curl http://localhost:8000/health # UgaJapa Bot

curl http://localhost:5000/health # Translation API

curl http://localhost:8065/api/v4/system/ping # Mattermost
```

12.2 Environment Variables (.env)

```
# Translation API Service

PORT=5000

DATABASE_URL=postgresql://mmuser:mmuser_password@localhost:5432/ugajapa_api

JWT_SECRET=your-very-long-random-secret-here

BOT_URL=http://localhost:8000


# Optional fallback engines

GOOGLE_API_KEY= # only needed if Google fallback is enabled

SUNBIRD_API_URL=https://api.sunbird.ai/translate
```

```
SUNBIRD_API_KEY=


# Billing cron

BILLING_CRON_SECRET=another-random-secret
```

12.3 Production VPS Deployment (Hetzner/DigitalOcean)

For a production deployment, both services run on the same VPS with NGINX as a reverse proxy. The UgaJapa Bot (port 8000) is **not** exposed publicly — only the Translation API (port 5000) is accessible through NGINX via HTTPS.

```
# NGINX config — Translation API only (bot stays internal)

server {

listen 443 ssl;

server_name api.ugajapa.ac.ug;


location / {

proxy_pass http://localhost:5000;

proxy_set_header X-Real-IP $remote_addr;

}

# Port 8000 (bot) is NOT proxied — internal only

}
```

13 How Each Repository Connects

The three repositories are completely independent codebases. They connect only at runtime through HTTP API calls. No repository imports code from another.

Connection	Direction	Protocol	Auth
RC Plugin → Translation API	Outbound from plugin	HTTP POST	X-API-Key header
MM Plugin → Translation API	Outbound from plugin	HTTP POST	X-API-Key header
Translation API → UgaJapa Bot	Internal only	HTTP POST	None (localhost)
Translation API → Sunbird AI	External (optional)	HTTP POST	Sunbird API key
Translation API → Google (fallback)	External (optional)	HTTP POST	Google API key
Translation API → PostgreSQL	Internal only	TCP/pg	DB credentials

14 Responding to Kuwatani-san's Requirements

This section maps every requirement stated in Kuwatani-san's messages of 6 July 2026 to the specific feature or component in this repository that satisfies it.

Kuwatani-san's Requirement	Satisfied By	Status
Separate source code for translation API into new repository	ugajapa-translation-api repo — this document	Ready
Unique connection key (API key) per user	POST /keys/generate — issues ugj_live_xxx per user	Designed
Plugin uses connection key to send requests	Mattermost plugin sends X-API-Key header on every /translate call	Already working
Translation quality evaluation in every response	quality: { levenshtein, semantic, label, color } in every /translate response	Designed
Quality evaluation differentiates from typical APIs	Back-translation scoring is not offered by Google/DeepL/MyMemory	Confirmed
Sign-up feature	POST /auth/signup	Designed
Login function	POST /auth/login → JWT token	Designed
API key issuance and management	POST /keys/generate, GET /keys, DELETE /keys/:id	Designed
Usage recording (characters translated)	usage_records table — every /translate call logged per key	Designed
Monthly billing based on usage	GET /billing/:month — cron calculates monthly totals	Designed
Architecture allowing easy switching of translation engines	router.ts — one file controls engine selection per language pair	Designed
Suggestions for additional features	UgaJapa Bot (own model), quality feedback loop, cultural hints, admin dashboard	Proposed

✓ All of Kuwatani-san's stated requirements are addressed in this design. The ugajapa-translation-api repository is ready to begin implementation. The first step is to create the GitHub repository and push the initial structure, then build Layer 3 (UgaJapa Bot) followed by the auth and key management endpoints.